

Prueba Práctica – Unidad IV

Validación, Gestión, Herramientas y Estándares de Requisitos

Caso: Sistema de Gestión de Pedidos

Sánchez Centeno Roselyn Andreina

4to Software “A”

Ingeniería de Requisitos (ISR-401)

Docente: Ing. Gleiston Guerrero, Mg.

Universidad Técnica Estatal de Quevedo

12 de agosto de 2026

Repositorio del proyecto: https://github.com/Roselyn15/Ev_Practica_Unidad_IV_RS

Caso práctico: Una tienda en línea requiere un módulo para gestionar los pedidos de sus clientes. Un *Cliente* (cédula/RUC, nombre, correo) puede realizar cero o más *Pedidos*. Cada *Pedido* (identificador, fecha, total) contiene una o más *Líneas de pedido*, y cada línea referencia un *Producto* (código, descripción, precio, stock). Al registrar un pedido, el sistema verifica la disponibilidad de stock: si hay existencias, confirma el pedido; si no, notifica el faltante. Un pedido confirmado puede despacharse, luego entregarse, o bien anularse mientras no haya sido entregado.

P1. Modelo de datos – Diagrama de clases UML

Se identifican las clases **Cliente**, **Pedido**, **LineaPedido** y **Producto**, con sus atributos tipados, operaciones y multiplicidades en los extremos de las asociaciones (**Cliente 1 - * Pedido**, **Pedido 1 - * LineaPedido**, **LineaPedido * - 1 Producto**).

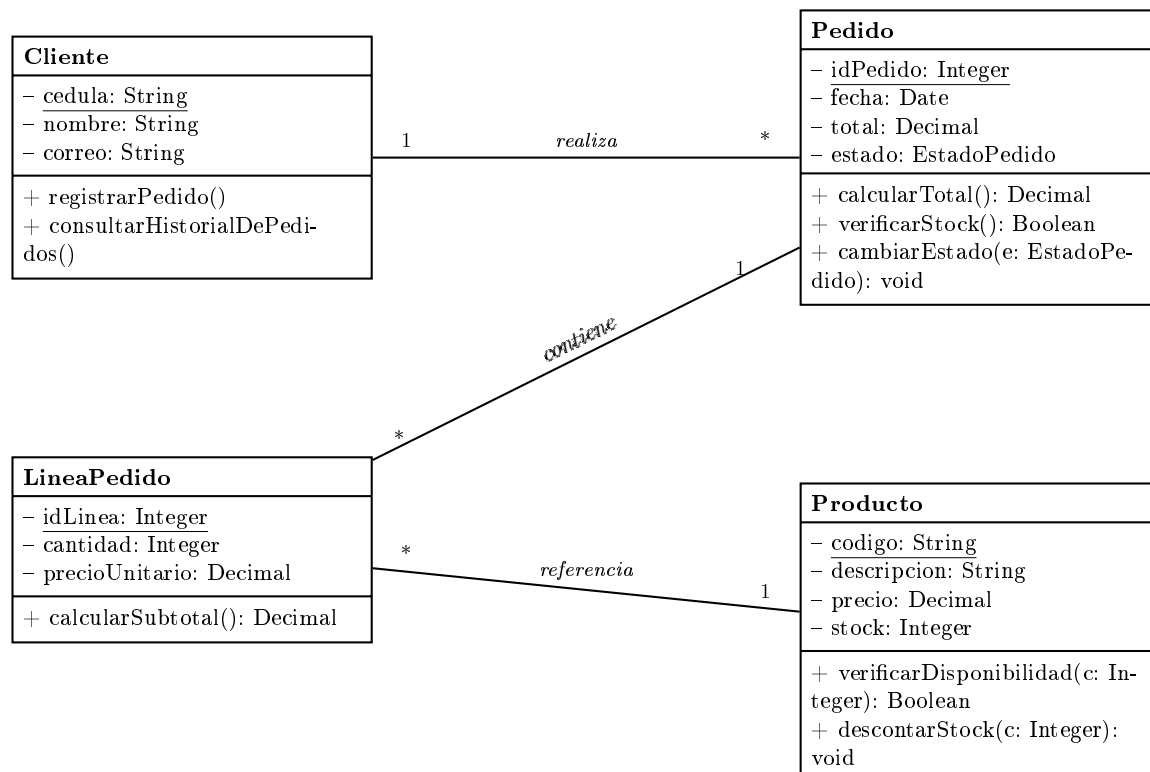


Figura 1: Diagrama de clases UML del dominio Sistema de Gestión de Pedidos (elaborado en TikZ, con asociaciones y multiplicidades explícitas).

P2. Modelo funcional – Diagrama de actividades UML

Se modela el proceso *Registrar pedido* usando dos carriles de responsabilidad: **Cliente** y **Sistema**. Incluye nodo inicial, decisión sobre disponibilidad de stock, los dos caminos alternativos (confirmar / notificar faltante), su convergencia y el nodo final.

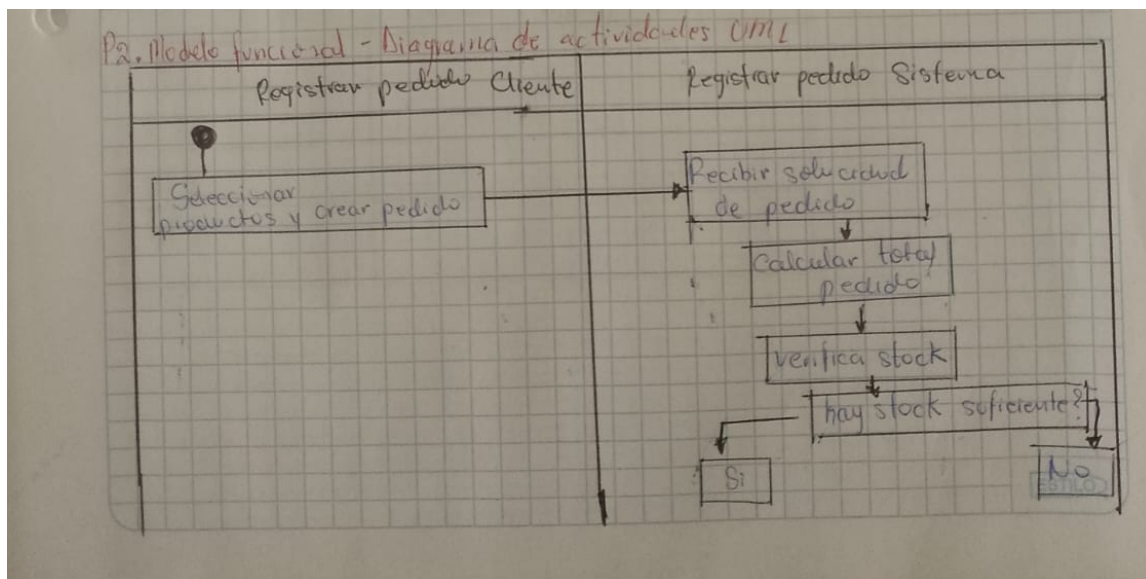


Figura 2: Diagrama de actividades UML – Registrar pedido (parte 1: solicitud, cálculo de total y verificación de stock).

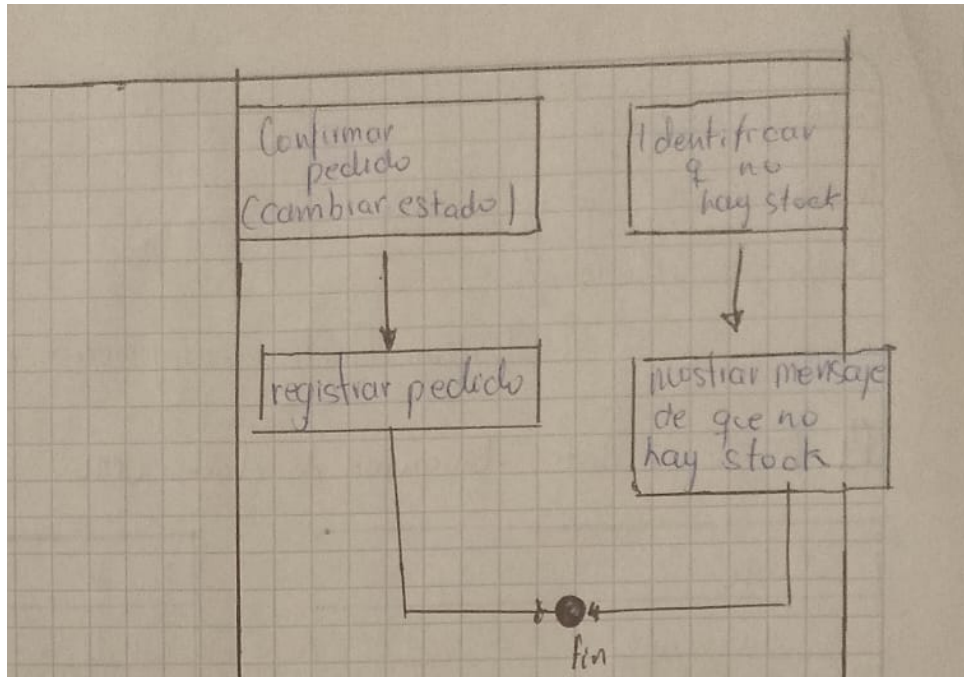


Figura 3: Diagrama de actividades UML – Registrar pedido (parte 2: continuación de los caminos “Sí” y “No” hasta el nodo final).

P3. Modelo de comportamiento – Máquina de estados UML

Ciclo de vida de un **Pedido**. Los estados **Confirmado** y **Enviado** se agrupan en el superestado **EnProceso**, de modo que la transición **anular** se factoriza una sola vez desde el borde del superestado hacia **Cancelado** (aplica mientras el pedido esté confirmado o enviado, pero no una vez entregado). La transición **Registrado** → **Confirmado** lleva la guarda **[stock disponible]**; la transición **Registrado** → **Cancelado** lleva la guarda **[stock no disponible]**. **Entregado** y **Cancelado** son los dos estados finales de aceptación del proceso.

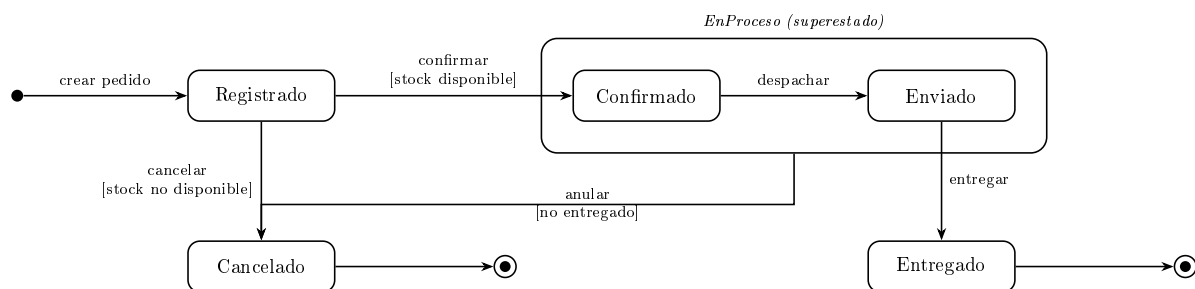


Figura 4: Máquina de estados UML del ciclo de vida de un Pedido, con superestado **EnProceso**, guardas y estados finales.

P4. Consistencia entre las tres perspectivas

Evento (P3)	Función/acción (P2)	Entidad/atributo (P1)
registrar	Recibir solicitud de pedido	Pedido (creación)
despachar	<i>sin función explícita en P2</i>	Pedido.estado = Enviado
confirmar	Confirmar pedido (cambiar estado)	Pedido.estado = Confirmado
cancelar	Identificar que no hay stock	Pedido.estado = Cancelado
entregar	<i>sin función explícita en P2</i>	Pedido.estado = Entregado

Cuadro 1: Tabla de verificación cruzada entre eventos, funciones y entidades.

Defecto detectado: los eventos **despachar** y **entregar** de la máquina de estados (P3) no tienen una función/acción correspondiente en el diagrama de actividades (P2), el cual solo modela el subproceso de *registro* del pedido. Esto constituye una inconsistencia entre los modelos, ya que P3 define transiciones para las cuales P2 no describe el comportamiento.

Corrección aplicada: se extiende el modelo funcional (P2) agregando las actividades faltantes *Despachar pedido* y *Entregar pedido* en el carril del Sistema, de modo que todos los eventos de la máquina de estados (P3) queden representados en el diagrama de actividades.

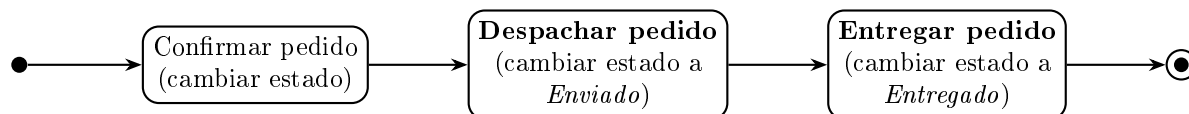


Figura 5: Fragmento corregido del diagrama de actividades (P2): actividades “Despachar pedido” y “Entregar pedido” añadidas en el carril del Sistema, cerrando la inconsistencia detectada.

P5. Especificación de requisitos con esquema de atributos

Requisitos funcionales

- RF-1** Los pedidos deben ser registrados en el sistema por el cliente, dado que existe stock suficiente de los productos solicitados.
- RF-2** Los datos personales del cliente (cédula, nombre, correo) registrados en un pedido deben mantenerse protegidos frente a accesos no autorizados.

Requisitos no funcionales

- RNF-1** *Eficiencia de desempeño (ISO/IEC 25010:2023)*: el sistema debe registrar un pedido en un tiempo de respuesta menor a 0.2 segundos.
- RNF-2** *Seguridad (ISO/IEC 25010:2023)*: el sistema debe cifrar los datos personales del cliente almacenados, garantizando su confidencialidad.

ID	Prioridad	Estado	Fuente	Estabilidad	Versión	Verificación
RF-1	Alta	Propuesto	Caso de negocio	Estable	1.0	Prueba funcional de registro
RF-2	Alta	Propuesto	Regulación de datos	Estable	1.0	Inspección de control de acceso
RNF-1	Media	Propuesto	Cliente	Estable	1.0	Prueba de rendimiento (cronometraje)
RNF-2	Alta	Propuesto	Normativa de seguridad	Estable	1.0	Auditoría de cifrado

Cuadro 2: Esquema de atributos de los requisitos.

P6. Priorización MoSCoW

R#	Must	Should	Could	Won't have	Justificación
RF-1	✓				Sin registro de pedidos no existe el producto mínimo viable.
RF-2	✓				Proteger datos personales es obligación legal y de confianza del cliente.
RNF-1		✓			Mejora la experiencia, pero el sistema es viable con una latencia algo mayor.
RNF-2			✓		Deseable reforzar el cifrado; puede incorporarse en una iteración posterior sin bloquear el lanzamiento.

Cuadro 3: Clasificación MoSCoW de los requisitos (RF-1 y RF-2 son Must have).

P7. Validación por inspección (checklist ISO/IEC/IEEE 29148)

Paso 1: identificación de defectos (sin corregir)

Requisito	Propiedad evaluada	Defecto detectado
RF-1	Completo	El tiempo/condición de disponibilidad de stock no queda del todo exacto ni cuantificado.
RF-2	Singular	No se especifica <i>cómo</i> deben estar protegidos los datos (mecanismo no definido).
RNF-1	Factible	Es claro lo que se pide, pero la meta de 0.2s puede ser difícil de sustentar sin más contexto técnico.
RNF-2	Verificable	Se asume que el cifrado ya estaría integrado en el sistema, sin un criterio de verificación explícito.

Cuadro 4: Defectos detectados por inspección (identificación separada de la corrección).

Paso 2: retrabajo (requisitos corregidos)

- RF-1'** El sistema debe registrar un pedido únicamente cuando exista stock suficiente para todos los productos solicitados, verificando la disponibilidad antes de confirmar.
- RF-2'** Los datos personales del cliente deben almacenarse cifrados en reposo (AES-256) y transmitirse mediante un canal seguro (TLS 1.2 o superior).
- RNF-1'** El sistema debe registrar un pedido en un tiempo de respuesta menor o igual a 0.2 segundos, medido bajo condiciones de carga normal (percentil 95).
- RNF-2'** El sistema debe cifrar los datos personales del cliente y este control debe verificarse mediante auditoría de seguridad semestral.

P8. Pruebas de aceptación trazadas

Prueba	Tipo	Entradas	Criterio de éxito	Traza a
PA-01	Funcional	Pedido con productos en stock	El pedido queda registrado y confirmado	RF-1
PA-02	Funcional	Solicitud de acceso a datos de un cliente	El acceso es denegado sin autorización	RF-2
PA-03	No funcional	Registro de 100 pedidos concurrentes	Tiempo de respuesta $\leq 0.2s$ en el 95% de los casos	RNF-1
PA-04	No funcional / Regresión	Inspección de la base de datos tras un despliegue	Los campos personales se almacenan cifrados	RNF-2

Cuadro 5: Pruebas de aceptación trazadas a cada requisito de P5.